TriSense-CenterPoint: Lidar–Radar–Camera Fusion for Long-Range 3D Object Detection on Autonomous Trucks

Author Name
Department
Affiliation
City, Country
email@example.com

***Abstract*—Autonomous trucks need to perceive objects at highway speeds and at ranges beyond those covered by most driving benchmarks, around a vehicle whose size and articulated trailer create large blind regions for any single sensor. We present TriSense-CenterPoint, a bird's-eye-view (BEV) detector that fuses a merged six-lidar point cloud, six 4D imaging radars and four cameras on the MAN TruckScenes dataset. The design starts from a voxel CenterPoint detector and adds a radar pillar branch and a lift-splat camera branch through *residual, zero-initialized* fusion blocks, so that the fused network initially reproduces the lidar-only detector exactly and each new modality can only add information. Radar Doppler measurements are ego-motion compensated with the truck's chassis velocity and yaw rate, kept consistent under geometric data augmentation, and used directly as supervision through a Doppler consistency loss on the predicted object velocity. The camera branch is supervised by lidar depth and a BEV heatmap, and a two-stage schedule first warms up the new branches with the lidar detector frozen, then trains all components jointly with modality dropout. Evaluated on the official TruckScenes test server, TriSense-CenterPoint reaches 0.554 NDS and 0.505 mAP under the challenge's sensor-history limits, with no test-time augmentation, future frames, maps or external data. This is the highest NDS and mAP among the results published on the evaluation server at the time of writing, ahead of the strongest lidar-only (0.551 NDS), camera–lidar (0.482) and radar–camera (0.420) entries. On the validation set it improves the lidar-only starting point from 0.433 to 0.548 NDS and lowers the mean velocity error from 2.01 to 0.77 m/s. We also analyse the remaining failure modes per class and per driving condition, which point to night, tunnels and fog as the open problems for truck perception.**

***Index Terms*—3D object detection, sensor fusion, 4D radar, lidar, autonomous trucking, bird's-eye view**

I. Introduction

Most progress in 3D object detection has been measured on passenger-car datasets such as KITTI [1], nuScenes [2] and the Waymo Open Dataset [3]. Autonomous trucks pose a different problem. A loaded semi-trailer truck needs a much longer distance to brake than a car, so the detection range that matters is longer, and the MAN TruckScenes benchmark [4] therefore evaluates vehicles up to 150 m away. The tractor and trailer occlude large parts of the scene for any single sensor, which is why trucks carry many sensors around the cab. The operating domain is also broader: highways at high speed, logistics terminals, rural roads, tunnels and bridges, in rain, fog and at night.

These conditions favour fusing complementary sensors. Lidar gives accurate geometry but its points become sparse at long range and degrade in fog. Cameras give dense appearance and semantics but weak depth. 4D imaging radar is robust to weather and lighting and measures radial velocity directly through the Doppler effect, but its point clouds are sparse and noisy. Velocity matters in particular for trucks: at highway speeds, an error of a few m/s in an object's velocity is a large error in where that object will be a second later.

In this paper we describe TriSense-CenterPoint, the detector we submitted to the MAN TruckScenes 3D detection benchmark. It fuses all three sensor types in a shared BEV grid on top of a strong lidar detector. Our contributions are:

- A tri-modal BEV detector for trucks that extends a voxel CenterPoint [5] with a radar pillar branch and a lift-splat-shoot (LSS) [6] camera branch through residual, zero-initialized fusion blocks. The fused model starts out *exactly* equal to the lidar detector, which makes training stable and lets new modalities be added without degrading what the lidar model already does.
- A physically consistent use of radar Doppler: ego-motion compensation with chassis velocity and yaw rate at each radar's mount point, augmentations that transform the velocity vectors and lines of sight together with the points, and a Doppler consistency loss that supervises the predicted object velocity with raw radial measurements.
- A two-stage training recipe (frozen-detector warm-up of all new branches with auxiliary heads and lidar depth supervision, followed by joint training with modality dropout).
- Results on the official test server (0.554 NDS, 0.505 mAP), the highest among the published server results, with a comparison against lidar-only, camera–lidar, radar–camera and radar-only entries and a per-class and per-condition analysis that identifies where truck perception still fails.

II. Related Work

Lidar 3D detection. VoxelNet [7] learned features from voxelized point clouds, and SECOND [8] made this practical with sparse 3D convolutions. PointPillars [9] encodes vertical pillars for a purely 2D backbone. CenterPoint [5] represents objects as BEV heatmap peaks and regresses their size, orientation and velocity, and together with class-balanced grouping and sampling (CBGS) [10] is a strong baseline on nuScenes. We build on the voxel CenterPoint in OpenPCDet [11].

Camera–lidar fusion in BEV. LSS [6] lifts image features into 3D with a per-pixel depth distribution and splats them onto a BEV grid; BEVDet [12] and BEVDepth [13] turned this into multi-camera detectors, the latter showing that explicit depth supervision from lidar is essential. BEVFusion [14], [15] fuses camera BEV features with lidar BEV features by concatenation and convolution, while TransFusion [16] fuses at the query level. Our camera branch follows the BEVFusion style, but we fuse residually from a zero initialization and add an auxiliary camera heatmap loss.

Radar fusion. CenterFusion [17] associates radar points with camera detections, and RadarNet [18] fuses radar and lidar both early and late, using radial velocity to refine object motion. CRN [19] and RCBEVDet [20] fuse radar and camera in BEV. 4D imaging radar datasets such as View-of-Delft [21] showed the value of elevation and Doppler for road-user detection. We use radar as a third modality next to lidar and cameras, and use its Doppler measurements both as an input feature and as a training signal.

Trucking datasets. MAN TruckScenes [4] is the first large public dataset for autonomous trucking. It provides six lidars, six 4D radars and four cameras, with annotations of 12 detection classes including trailers, traffic signs and animals, and scene tags for weather, area, time of day, lighting and road structure. Its benchmark uses nuScenes-style metrics with class-specific evaluation ranges of up to 150 m.

III. Dataset and Task

TruckScenes scenes are 20 s long and annotated at 2 Hz keyframes. We use the official split: 20,907 training and 2,995 validation keyframes, and 5,960 test keyframes in 149 scenes, whose annotations are hidden behind an evaluation server. Detections are scored with the official `truckscenes-devkit`: mean average precision (mAP) over center-distance thresholds of 0.5, 1, 2 and 4 m, five true-positive (TP) errors for translation (mATE), scale (mASE), orientation (mAOE), velocity (mAVE) and attributes (mAAE), and the nuScenes detection score

$$\mathrm{NDS} = \tfrac{1}{10}\Big[\,5\,\mathrm{mAP} + \textstyle\sum_{e\in\mathrm{TP}}\big(1-\min(1,e)\big)\Big]. \qquad (1)$$

Predictions and annotations are filtered by class-specific distance: 150 m for vehicles, 75 m for the other classes.

The benchmark limits the sensor history to about 0.5 s at inference: at most 6 past lidar sweeps, 11 past radar sweeps and 6 past camera images. A submission may contain at most 500 boxes per keyframe, and it must declare which sensors, maps, external data, future frames and test-time augmentation it used.

Conversion to the nuScenes format. To reuse mature nuScenes tooling, we convert TruckScenes into nuScenes tables. The six lidars of every frame are merged into one virtual, ego-aligned top lidar. Each sensor's points are transformed sensor $\rightarrow$ ego(t_s) $\rightarrow$ global $\rightarrow$ ego(t_{ref}) with the ego poses at their own timestamps, which compensates the up to 50 ms firing offsets between the lidars. Radar and camera data are read directly from the original dataset, together with their calibrations and ego poses. Evaluation always runs against the original TruckScenes tables, so the scores are directly comparable to the leaderboard.

IV. Method

Fig. 1 gives an overview. Three branches produce BEV feature maps on a common grid: 256×256 cells of 1.2 m, covering ± 153.6 m around the truck, enough for the 150 m evaluation range. Two residual fusers combine them, and a shared 2D backbone and CenterPoint head predict the boxes.

A. Lidar Branch

The merged lidar cloud of the keyframe and its 4 most recent past sweeps (well within the limit of 6) is voxelized at $0.15 \times 0.15 \times 0.2$ m over ± 153.6 m horizontally and $[-4, 4]$ m vertically. Each point carries its coordinates, intensity and time lag to the keyframe, and each voxel the mean of its points. A sparse 3D residual backbone (VoxelResBackBone8x) with stride 8 yields the 256-channel lidar BEV map F_L after height compression. This branch and the detector head are those of a lidar-only CenterPoint trained on TruckScenes, which is also our starting point.

B. Radar Branch

Ego-motion compensation. TruckScenes stores each radar return with its raw Doppler velocity, a vector $\mathbf{v}^{\mathrm{rel}}$ along the line of sight in the sensor frame, which is not compensated for the truck's own motion. A truck rotates noticeably while turning, and its radars are mounted metres away from the ego origin, so the sensor's velocity differs from the chassis velocity. For a radar mounted at $\mathbf{t}_s$ in the ego frame with rotation R_s, we compute the velocity of the mount point from the chassis velocity $\mathbf{v}_e$ and yaw rate ω,

$$\mathbf{v}_s = R_s^{\top}\big(\mathbf{v}_e + \omega\,\hat{\mathbf{z}} \times \mathbf{t}_s\big), \qquad (2)$$

and the compensated radial velocity of a return with unit ray $\hat{\mathbf{r}}$ (sensor frame) as

$$v_r = \mathbf{v}^{\mathrm{rel}} \cdot \hat{\mathbf{r}} + \mathbf{v}_s \cdot \hat{\mathbf{r}}, \qquad (3)$$

which is zero for static objects. Each sample uses the keyframe and the 6 most recent past sweeps of each of the 6 radars (within the limit of 11). Their points are moved into the reference frame at the keyframe time using the ego poses. Each point has the features $(x, y, z, \mathrm{RCS}, v_r\hat{r}_x, v_r\hat{r}_y, v_r, \Delta t)$, and we also keep its ray and range for the augmentations and the Doppler loss.

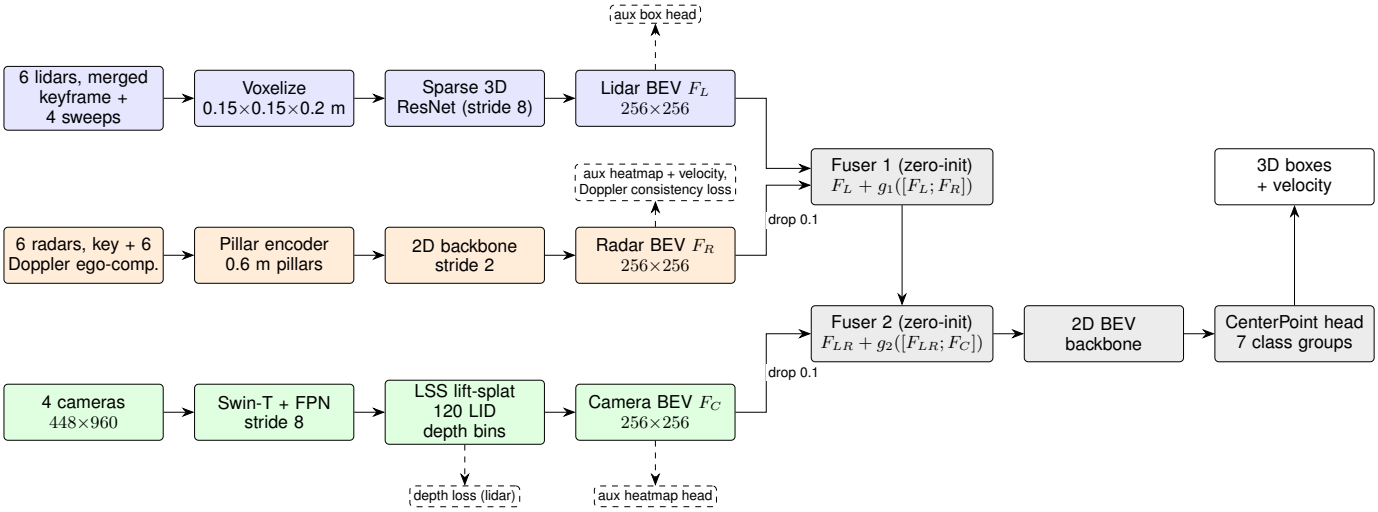


Fig. 1. Overview of TriSense-CenterPoint. Lidar, radar and camera features are built on the same 256×256 BEV grid with 1.2 m cells covering ±153.6 m. Radar and camera BEV maps are added through residual fusers whose last batch-norm scale is initialized to zero, so the network starts as the lidar-only CenterPoint. Dashed: losses used only during training. Modality dropout zeroes the radar or camera BEV of a sample with probability 0.1.

Equivariant augmentation. The usual global flips, rotations and scalings of lidar detectors would make radar velocities inconsistent with the scene if applied only to point positions. We apply each transform to the positions, the radial velocity vectors and the rays together: flips negate the corresponding components, rotations rotate all three, and scaling scales positions, ranges and velocities by the same factor as the box velocities.

Encoder. A PointPillars-style encoder [9] with 0.6 m pillars and 64 channels, followed by a two-stage 2D backbone with stride 2, gives the radar BEV map F_R on the lidar grid. The whole radar branch has only 0.7 M parameters.

C. Camera Branch

The four cameras (left and right, front and back) have a resolution of 1980×943. Each image is resized by 0.485 and cropped to 448×960, mostly removing sky. An ImageNet-pretrained Swin-T [22] backbone and an FPN [23] neck give 256-channel features at stride 8 (56×120). An LSS view transform [6] predicts, for every feature cell, a categorical depth distribution over $D = 120$ bins and an 80-channel context feature, and splats their outer product onto a 0.6 m BEV grid that is pooled to the 1.2 m lidar grid. For a range of 150 m, uniform bins are either too coarse near the truck or too many in total. We use linear-increasing discretization (LID) [24] instead, with bin edges

$$d_i = d_{\min} + (d_{\max} - d_{\min}) \, \frac{i(i+1)}{D(D+1)}, \quad i = 0, \ldots, D, \tag{4}$$

with $d_{\min} = 1$ m and $d_{\max} = 153.6$ m. This gives bins of about 2 cm next to the truck and about 2.5 m at the far end. The camera branch is camera-only: unlike some BEVFusion variants, lidar depth is never an input to it, only a training target. The back cameras fire about 35 ms away from the lidar, so each camera-to-lidar transform uses the ego pose at that camera's own timestamp.

D. Residual Zero-Initialized Fusion

Both fusion steps use the same block. Given a base BEV map F_A and an added map F_B on the same grid,

$$F' = F_A + g\big([F_A; F_B]\big), \tag{5}$$

where g is a 3×3 convolution, batch norm and ReLU to 256 channels, followed by a 3×3 convolution back to the channels of F_A and a batch norm whose scale γ is initialized to zero. We first fuse radar into lidar, $F_{LR} = F_L + g_1([F_L; F_R])$, and then the camera into the result, $F_{LRC} = F_{LR} + g_2([F_{LR}; F_C])$. The shared 2D backbone and the CenterPoint head then work on F_{LRC}.

Because $\gamma = 0$, the fused network computes exactly the lidar detector when it is initialized from a lidar checkpoint, whatever the random weights of the new branches. We verified this: after the warm-up stage, in which the fusers are not trained, the model reproduces the lidar-only validation mAP to the fourth decimal (Table III). Training can therefore only move away from a good detector, rather than recover from the noise that randomly initialized branches inject into the shared backbone.

E. Detection Head and Auxiliary Supervision

The detection head is the CenterPoint head [5], with seven class groups: {car}, {truck, other vehicle}, {bus, trailer}, {barrier}, {motorcycle, bicycle}, {pedestrian, traffic cone, animal} and {traffic sign}. It predicts a heatmap per class and regresses the center offset, height, size, orientation and the 2D velocity. We train it with the focal heatmap loss and an L_1 regression loss $\mathcal{L}_{\text{det}}$.

Fusion alone gives a branch no direct reason to learn useful features, especially when the fused output starts out ignoring it. We therefore add training-only auxiliary CenterPoint heads on each branch's BEV map *before* fusion: a full box head on F_L, a heatmap and velocity head on F_R, and a heatmap-only

head on F_C. Camera BEV positions are initially too coarse for box regression. None of these heads are run at inference.

Doppler consistency loss. Radar measures the projection of an object's velocity onto each line of sight. For every ground-truth box b with radar points $\mathcal{P}_b$ (inside the box enlarged by 1 m horizontally and 2 m vertically), we take the velocity $\hat{\mathbf{v}}_b$ predicted by the radar auxiliary head at the BEV cell of the box center, and penalize

$$\mathcal{L}_{\mathrm{Dop}} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \frac{1}{|\mathcal{P}_b|} \sum_{p \in \mathcal{P}_b} \mathrm{SmoothL1}\big(\hat{\mathbf{v}}_b \cdot \hat{\mathbf{r}}_p^{xy} - v_{r,p}\big), \quad (6)$$

where $\mathcal{B}$ are the boxes with at least one radar point. The loss is averaged per box, so large trucks with many returns do not dominate. This ties the predicted velocity to the actual measurements rather than only to annotated velocities, which are derived from box positions in consecutive frames.

Depth supervision. Following BEVDepth [13], we project the current lidar sweep into each augmented image and keep, for every stride-8 feature cell, the nearest lidar depth. Cells with lidar depth are supervised with a binary cross-entropy between the predicted depth distribution and the one-hot depth bin. The total loss is

$$\mathcal{L} = \mathcal{L}_{\mathrm{det}} + 0.5\big(\mathcal{L}_{\mathrm{aux}}^{L} + \mathcal{L}_{\mathrm{aux}}^{R} + \mathcal{L}_{\mathrm{aux}}^{C}\big) + 0.2\,\mathcal{L}_{\mathrm{Dop}} + 3\,\mathcal{L}_{\mathrm{depth}}. \quad (7)$$

F. Two-Stage Training

Lidar pre-training. We first train the lidar-only CenterPoint with CBGS [10], GT sampling and global augmentations (batch 16, Adam [25] with a one-cycle schedule [26], peak learning rate 5×10^{-4}). We use its checkpoint after 4 epochs.

Stage A (warm-up). All lidar modules, both fusers, the shared 2D backbone and the detection head are frozen, with their batch-norm statistics fixed. Only the radar and camera branches and the three auxiliary heads train, on the auxiliary, Doppler and depth losses, for one CBGS epoch (batch 8, peak learning rate 10^{-3}). The fusers and the detection head are not run, so the warm-up costs little memory, and the detector stays exactly the lidar detector. Without this stage, freshly initialized auxiliary heads produce heatmap losses in the thousands in the first iterations, and their gradients would reach the trained lidar backbone.

Stage B (joint). All parameters are trained with the full loss for 6 CBGS epochs (batch 6, one-cycle with peak learning rate 7.5×10^{-4}, weight decay 0.01, gradient clipping at 10). With modality dropout, the radar or camera BEV of each sample is zeroed with probability 0.1 before its fuser, so the detector does not depend on a single added sensor. Lidar is never dropped, since every benchmark sample has lidar.

Augmentation. Stage B uses global flips along both axes, rotations in $\pm 45°$, scaling in $[0.9, 1.1]$ and translation noise ($\sigma = 0.5$ m), applied consistently to lidar, radar, boxes and the camera extrinsics. Each image is also flipped, rotated in $\pm 5.4°$ and randomly resized and cropped, with the exact image-augmentation matrix kept for the view transform. We do not use GT sampling in the fused model, because pasted lidar and radar objects would have no matching image content.

Implementation. The code is built on OpenPCDet [11] and trained with mixed precision on NVIDIA A100 (80 GB) GPUs. The camera auxiliary head is kept in FP32, because its heatmap convolutions overflowed FP16 during the warm-up. The inference model (without auxiliary heads) has 44.5 M parameters, of which 27.5 M are in the Swin-T backbone.

G. Inference

At test time the auxiliary heads, depth loss and modality dropout are disabled. The head keeps boxes with score above 0.1 and applies per-group NMS (IoU 0.2). We then keep the 500 highest-scoring boxes per keyframe over all groups, because the benchmark rejects submissions with more. Attributes are assigned from the predicted velocity (moving if faster than 0.2 m/s) and class, and classes without attributes (barrier, traffic cone) get none. No test-time augmentation is used. Running the full pipeline, including data loading, on the 5,960 test keyframes takes about 37 minutes on one A100 with batch size 4.

V. Experiments

A. Test Set Results

Our final model (stage B, epoch 6) was evaluated on the hidden test set under the benchmark's rules: inputs limited to 4 past lidar sweeps, 6 past radar sweeps and the current camera images, and no maps, external data, future frames or test-time augmentation. It reaches 0.505 mAP and 0.554 NDS (Table I), slightly above its validation scores (0.492 mAP, 0.548 NDS), so it does not appear overfitted to the validation set on which we selected the checkpoint. The TP errors are similar on both splits: orientation error is lower on test (0.28 versus 0.38) and velocity error is higher (0.87 versus 0.77 m/s).

B. Comparison with Other Methods

Table I compares TriSense-CenterPoint with all methods whose results are published on the evaluation server. It has the highest NDS and mAP, and the lowest translation and orientation errors. Three groups of entries stand out.

Lidar-based methods. The strongest other entry is an unnamed lidar-only submission (0.551 NDS). Our NDS margin over it is small (+0.003), but our mAP is 2.4 points higher. Table II shows that the two models have different strengths. TriSense-CenterPoint is far better on the rare and large classes: other vehicles (0.254 versus 0.002 AP), buses (0.401 versus 0.190) and barriers (0.565 versus 0.443). The lidar-only model is better on the small, frequent classes (pedestrians, traffic cones, signs) and on cars, and its velocity error is lower (0.76 versus 0.87 m/s). This suggests that a better-tuned lidar branch, for example with longer training or GT sampling, could be combined with our fusion to gain on both sides. Compared with the CenterPoint entry, which uses the same detector family with lidar only, our model scores 0.145 higher NDS and 0.238 higher mAP. Part of this gap is due to training and tuning, not only to fusion.

Camera–lidar fusion. The best camera–lidar entry has a similar mAP (0.485) but a much lower NDS (0.482), because

TABLE I

COMPARISON ON THE TRUCKSCENES TEST SET. ALL ENTRIES ARE RESULTS PUBLISHED ON THE OFFICIAL EVALUATION SERVER (ACCESSED NOVEMBER 2, 2026), WITH THE SENSORS EACH SUBMISSION DECLARED. FOR METHODS WITH SEVERAL SUBMISSIONS WE LIST THE BEST ONE; DEBUGGING AND DUMMY SUBMISSIONS ARE OMITTED. L: LIDAR, R: RADAR, C: CAMERA. †: DECLARED USE OF EXTERNAL DATA. LOWER IS BETTER FOR THE TP ERRORS.

#	Method	L	R	C	NDS↑	mAP↑	mATE	mASE	mAOE	mAVE	mAAE
1	**TriSense-CenterPoint (ours)**	✓	✓	✓	**0.554**	**0.505**	**0.373**	0.359	**0.276**	0.868	0.108
2	Unnamed submission (Aug. 2026)	✓			0.551	0.481	0.377	0.373	0.293	**0.763**	**0.088**
3	Unnamed submission (Mar. 2026)	✓		✓	0.482	0.485	0.428	0.381	0.542	2.240	0.255
4	SpaRC [27]		✓	✓	0.420	0.332	0.717	0.393	0.355	0.784	0.208
5	CenterPoint [5]	✓			0.410	0.267	0.409	**0.352**	0.277	2.730	0.201
6	RaGSplat		✓		0.236	0.161	0.680	0.697	0.594	0.897	0.573
7	HyDRa† [28]		✓	✓	0.224	0.128	0.725	0.544	0.744	1.180	0.388
8	DRaGAN		✓		0.209	0.088	0.728	0.639	0.644	1.044	0.339
9	PETR [29]*			✓	0.121	0.022	1.125	0.686	0.647	1.499	0.564
10	RadarGNN [30]		✓		0.108	0.070	0.892	0.809	1.132	8.003	0.571

*PETR is a camera-only method; its submission declared lidar instead of camera input.

TABLE II

TEST AP PER CLASS OF THE STRONGEST ENTRIES OF TABLE I. #2: UNNAMED LIDAR-ONLY, #3: UNNAMED CAMERA–LIDAR SUBMISSION.

Class	Ours	#2 (L)	#3 (L+C)	SpaRC	CenterPoint
Car	0.842	**0.871**	0.822	0.581	0.329
Truck	0.588	0.587	**0.591**	0.392	0.227
Bus	**0.401**	0.190	0.372	0.148	0.055
Trailer	0.652	**0.665**	0.647	0.494	0.336
Other vehicle	**0.254**	0.002	0.024	0.089	0.138
Pedestrian	0.584	**0.674**	0.579	0.279	0.451
Motorcycle	0.598	**0.620**	0.576	0.373	0.222
Bicycle	0.136	0.165	**0.256**	0.222	0.064
Traffic cone	0.720	**0.781**	0.758	0.553	0.547
Barrier	**0.565**	0.443	0.451	0.347	0.415
Animal	0.000	0.000	0.000	0.000	0.000
Traffic sign	0.719	**0.770**	0.745	0.510	0.423

its velocity (2.24 m/s) and orientation (0.54 rad) errors are large. Our model lowers the velocity error by 61% and roughly halves the orientation error, which is consistent with radar Doppler adding the motion information that neither lidar nor cameras measure directly.

Radar–camera and radar-only methods. Without lidar, the best method, SpaRC [27], reaches 0.420 NDS. Its velocity error (0.78 m/s) is close to ours, which confirms the value of radar for motion. However, its translation error is almost twice ours (0.72 versus 0.37 m), the expected cost of missing lidar geometry at long range. Radar-only methods stay below 0.24 NDS. This shows that radar cannot replace lidar for long-range truck perception, but it is a valuable addition to it.

C. Effect of Fusion and Training Progress

Table III and Fig. 2 follow the model through training on the validation set. After the warm-up, the network is still the lidar detector (0.334 mAP, 0.433 NDS), as intended. Joint training then raises mAP by 0.158 and NDS by 0.115. The largest single improvement is in velocity: mAVE falls steadily from 2.01 to 0.77 m/s, a 62% reduction, while translation and orientation errors fall by 17% and 18%. Scale error rises from 0.33 to 0.38. We attribute this to the dropped GT sampling, which removes the many pasted, well-sampled objects that the

TABLE III

VALIDATION RESULTS. THE LIDAR ROW IS THE STARTING CHECKPOINT OF BOTH STAGES. STAGE A MUST EQUAL IT, SINCE THE ZERO-INITIALIZED FUSERS ARE NOT TRAINED IN STAGE A.

Model	mAP↑	NDS↑	mATE	mASE	mAOE	mAVE
Lidar CenterPoint (ep. 4)	0.334	0.433	0.430	0.327	0.461	2.011
+ Stage A (warm-up)	0.334	0.433	0.429	0.327	0.469	2.011
Stage B, epoch 2	0.417	0.466	0.460	0.400	0.443	1.630
Stage B, epoch 3	0.482	0.504	0.429	0.400	0.424	1.281
Stage B, epoch 4	0.472	0.505	0.394	0.388	0.424	1.060
Stage B, epoch 5	**0.497**	0.545	0.359	0.374	0.380	0.823
Stage B, epoch 6 (final)	0.492	**0.548**	**0.358**	0.380	**0.377**	**0.774**

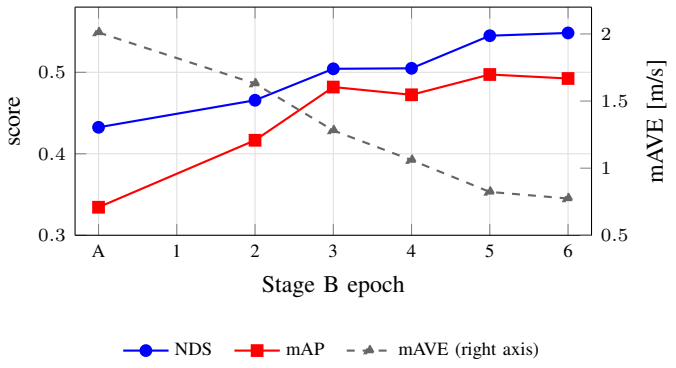


Fig. 2. Validation NDS, mAP and velocity error over training. "A" is the model after the warm-up stage, which equals the lidar-only starting point. Epoch 1 was not evaluated.

lidar model saw in training. The two last epochs have nearly the same scores, so we submitted the last one, which has the best NDS and the lowest TP errors.

Caveat on attribution. These gains combine the effect of the added sensors with that of seven further epochs of training, the last six with a fully annealed learning rate, while the lidar checkpoint was taken early in a 20-epoch schedule. They therefore give an upper bound on the contribution of fusion, not a controlled ablation. The size and direction of some changes still point to the added modalities. The velocity error,

TABLE IV
AP PER CLASS (MEAN OVER DISTANCE THRESHOLDS). LIDAR: STARTING CHECKPOINT ON VALIDATION; OURS: FINAL MODEL.

Class	Lidar (val)	Ours (val)	Ours (test)
Car	0.760	0.830	0.842
Truck	0.473	0.588	0.588
Bus	0.124	0.423	0.401
Trailer	0.545	0.667	0.652
Other vehicle	0.101	0.197	0.254
Pedestrian	0.512	0.584	0.584
Motorcycle	0.103	0.647	0.598
Bicycle	0.095	0.444	0.136
Traffic cone	0.574	0.729	0.720
Barrier	0.139	0.111	0.565
Animal	0.000	0.000	0.000
Traffic sign	0.586	0.687	0.719
mAP	0.334	0.492	0.505

TABLE V
TEST NDS / MAP PER SCENE TAG, AS REPORTED BY THE EVALUATION SERVER. CONDITIONS ABSENT FROM THE TEST SET (SNOW, HAIL, SPRING, PARKING AREAS) ARE OMITTED.

	Tag	NDS	mAP		Tag	NDS	mAP
Weather	clear	0.534	0.486	Lighting	illuminated	0.553	0.509
	overcast	0.510	0.445		twilight	0.342	0.280
	rain	0.426	0.419		dark	0.332	0.269
	fog	0.275	0.281		glare	0.294	0.277
Area	highway	0.472	0.441	Structure	regular	0.560	0.527
	terminal	0.415	0.405		bridge	0.467	0.374
	rural	0.395	0.411		underpass	0.426	0.395
	city	0.378	0.287		tunnel	0.220	0.210
	residential	0.368	0.267				
Daytime	morning	0.543	0.476	Season	summer	0.582	0.577
	noon	0.541	0.500		winter	0.473	0.425
	evening	0.331	0.287		autumn	0.443	0.366
	night	0.000	0.000				
Roadworks: 0.369 / 0.388				No roadworks: 0.546 / 0.493			

which radar measures directly and our Doppler loss supervises, falls the most. The classes that gain most are those that lidar sees with few points at long range (bus: 0.12→0.42 AP, motorcycle: 0.10→0.65, bicycle: 0.10→0.44, Table IV). We discuss the missing ablations in Section V-G.

D. Per-Class Results

Table IV breaks AP down by class. On validation, the fused model improves every class except barrier (0.139→0.111) and animal, which no model detects. Large vehicles benefit strongly: cars, trucks and trailers gain 7–12 AP points, and buses more than triple their AP.

Classes with few validation instances are very sensitive to the evaluated split and checkpoint. Barrier AP is 0.11 on validation but 0.57 on test, and bicycle AP is 0.44 on validation but 0.14 on test. Even for the lidar model alone, motorcycle AP moved from 0.40 at epoch 3 to 0.10 at epoch 4 of the same run. Results for these classes should be read as noisy, and the mean over 12 classes is noticeably affected by them. The animal class, with only a handful of annotations, remains at zero AP. It would need targeted resampling or a dedicated head.

E. Performance under Driving Conditions

The test server also reports scores per scene tag (Table V). Each tag subset is evaluated on its own, with its own class mix, so a subset can score lower than the full test set even if every subset is detected equally well. With that caveat, the breakdown shows clear trends:

- **Lighting and time of day** matter most. Twilight, dark and glare scenes score 0.29–0.34 NDS against 0.55 in illuminated scenes, and the night subset scores zero. Since lidar and radar do not depend on light, the zero on night scenes more likely reflects a very small or unusual subset (for example, only rare classes) than a complete failure of the detector, but it shows that the camera branch, trained on few dark images, contributes little there.
- **Fog** (0.28 NDS) is the hardest weather condition, consistent with lidar's sensitivity to fog. It suggests that the radar branch is not yet weighted strongly enough when lidar degrades. Modality dropout of *lidar* features, which we did not use, may help.
- **Tunnels** (0.22 NDS) are the hardest road structure. They combine poor lighting with enclosing walls, which plausibly cause radar multipath reflections and clutter in the lidar data; the tunnel subset is also small. Bridges and underpasses are less affected.
- **Urban scenes** (city, residential: 0.37 NDS) are harder than highways (0.47). They contain more pedestrians, cyclists and occlusions, while highway scenes are dominated by vehicles, which the model detects best.

F. Discussion: Velocity Estimation

Velocity remains the largest TP error (0.87 m/s on test). Object speeds on highways are high, our lidar history is short (0.4 s), and annotated velocities are finite differences of box positions, which makes them noisy. Doppler is the most direct velocity signal, but our model applies the Doppler loss only to the radar branch. Applying it to the fused head is a natural next step.

G. Limitations

Our study has several limitations. (i) As discussed above, we do not have controlled ablations. The comparison with the lidar model mixes fusion with additional training, and we did not train lidar+radar or lidar+camera variants with the same schedule. The individual contributions of the Doppler loss, the depth loss, the auxiliary heads and modality dropout are therefore not isolated. (ii) All numbers come from a single training run, so we cannot give variance estimates, which matter for the rare classes. (iii) Images are processed at about half resolution, which limits long-range camera detections. (iv) The fused model does not use GT sampling, which costs scale accuracy and may cost recall on rare classes. A multi-modal GT sampler that also pastes image patches could recover both.

VI. Conclusion

We presented TriSense-CenterPoint, a lidar–radar–camera BEV detector for autonomous trucks. Residual, zero-initialized fusion lets radar and camera branches be added to a trained lidar detector without ever degrading it. A physically consistent treatment of radar Doppler, from ego-motion compensation to a Doppler consistency loss, supports velocity estimation, and a two-stage training recipe keeps the joint training stable. On the MAN TruckScenes test set, the model reaches 0.554 NDS and 0.505 mAP under the benchmark's sensor-history limits, the best result published on the evaluation server, with the largest margins on rare vehicle classes and on velocity and orientation compared with camera–lidar fusion. The per-condition analysis shows that night, glare, fog and tunnels are still far from solved. Making the detector shift its weight towards radar when light or visibility degrades is, in our view, the most important direction for future work on truck perception.

Acknowledgment

We thank the MAN TruckScenes team for releasing the dataset and maintaining the evaluation server.

References

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3354–3361.

[2] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.

[3] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2446–2454.

[4] F. Fent, F. Kuttenreich, F. Ruch, F. Rizwin, S. Juergens, L. Lechermann, C. Nissler, A. Perl, U. Voll, M. Yan, and M. Lienkamp, "MAN TruckScenes: A multimodal dataset for autonomous trucking in diverse conditions," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[5] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3D object detection and tracking," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 11 784–11 793.

[6] J. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3D," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 194–210.

[7] Y. Zhou and O. Tuzel, "VoxelNet: End-to-end learning for point cloud based 3D object detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4490–4499.

[8] Y. Yan, Y. Mao, and B. Li, "SECOND: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, 2018.

[9] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "PointPillars: Fast encoders for object detection from point clouds," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 12 697–12 705.

[10] B. Zhu, Z. Jiang, X. Zhou, Z. Li, and G. Yu, "Class-balanced grouping and sampling for point cloud 3D object detection," *arXiv preprint arXiv:1908.09492*, 2019.

[11] OpenPCDet Development Team, "OpenPCDet: An open-source toolbox for 3D object detection from point clouds," https://github.com/open-mmlab/OpenPCDet, 2020.

[12] J. Huang, G. Huang, Z. Zhu, Y. Ye, and D. Du, "BEVDet: High-performance multi-camera 3D object detection in bird-eye-view," *arXiv preprint arXiv:2112.11790*, 2021.

[13] Y. Li, Z. Ge, G. Yu, J. Yang, Z. Wang, Y. Shi, J. Sun, and Z. Li, "BEVDepth: Acquisition of reliable depth for multi-view 3D object detection," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, no. 2, 2023, pp. 1477–1485.

[14] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. L. Rus, and S. Han, "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2023, pp. 2774–2781.

[15] T. Liang, H. Xie, K. Yu, Z. Xia, Z. Lin, Y. Wang, T. Tang, B. Wang, and Z. Tang, "BEVFusion: A simple and robust LiDAR-camera fusion framework," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[16] X. Bai, Z. Hu, X. Zhu, Q. Huang, Y. Chen, H. Fu, and C.-L. Tai, "TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 1090–1099.

[17] R. Nabati and H. Qi, "CenterFusion: Center-based radar and camera fusion for 3D object detection," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2021, pp. 1527–1536.

[18] B. Yang, R. Guo, M. Liang, S. Casas, and R. Urtasun, "RadarNet: Exploiting radar for robust perception of dynamic objects," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 496–512.

[19] Y. Kim, J. Shin, S. Kim, I.-J. Lee, J. W. Choi, and D. Kum, "CRN: Camera radar net for accurate, robust, efficient 3D perception," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 17 615–17 626.

[20] Z. Lin, Z. Liu, Z. Xia, X. Wang, Y. Wang, S. Qi, Y. Dong, N. Dong, L. Zhang, and C. Zhu, "RCBEVDet: Radar-camera fusion in bird's eye view for 3D object detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 14 928–14 937.

[21] A. Palffy, E. Pool, S. Baratam, J. F. P. Kooij, and D. M. Gavrila, "Multi-class road user detection with 3+1D radar in the View-of-Delft dataset," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4961–4968, 2022.

[22] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 10 012–10 022.

[23] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.

[24] C. Reading, A. Harakeh, J. Chae, and S. L. Waslander, "Categorical depth distribution network for monocular 3D object detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 8555–8564.

[25] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[26] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, ser. Proc. SPIE, vol. 11006, 2019, p. 1100612.

[27] P. Wolters, J. Gilg, T. Teepe, F. Herzog, F. Fent, and G. Rigoll, "SpaRC: Sparse radar-camera fusion for 3D object detection," *arXiv preprint arXiv:2411.19860*, 2024.

[28] P. Wolters, J. Gilg, T. Teepe, F. Herzog, A. Laouichi, M. Hofmann, and G. Rigoll, "Unleashing HyDRa: Hybrid fusion, depth consistency and radar for unified 3D perception," *arXiv preprint arXiv:2403.07746*, 2024.

[29] Y. Liu, T. Wang, X. Zhang, and J. Sun, "PETR: Position embedding transformation for multi-view 3D object detection," in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 531–548.

[30] F. Fent, P. Bauerschmidt, and M. Lienkamp, "RadarGNN: Transformation invariant graph neural network for radar-based perception," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, pp. 182–191.